COMP1531
Software Engineering Fundamentals

Week 4

Full-Stack - HTTP Servers

In This Lecture

– – –

Why?

- Web servers are fundamental part of web-based full-stack software

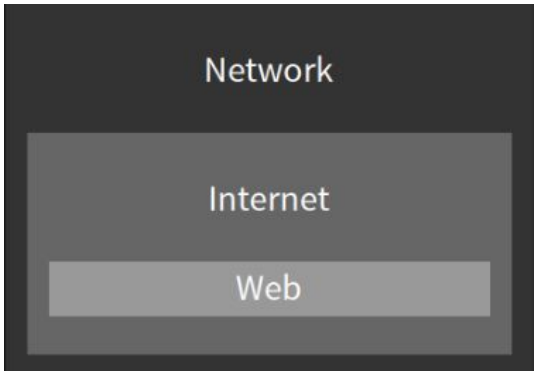
What?

- Networks
- Express Server
- APIs
- CRUD

Networks

– – –

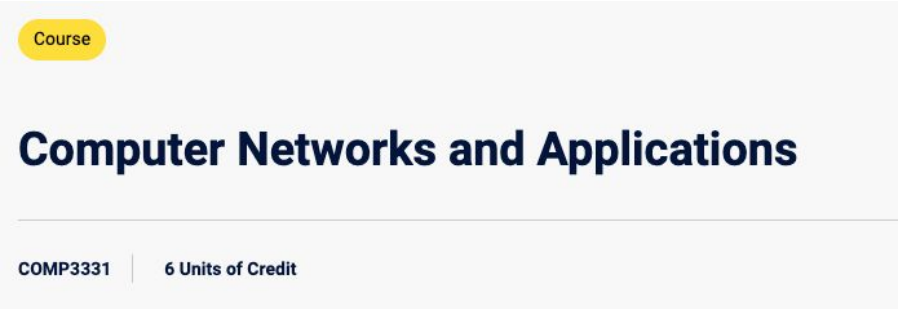
- **Network**: A group of interconnected computers that can communicate
- **Internet**: A global infrastructure for networking computers around the entire world together
- **World Wide Web**: A system of documents and resources linked together, accessible via URLs



Networks

So much to learn, it could be its own course!

- It is!
  - If you want to study networks, take COMP3331



Network Protocols

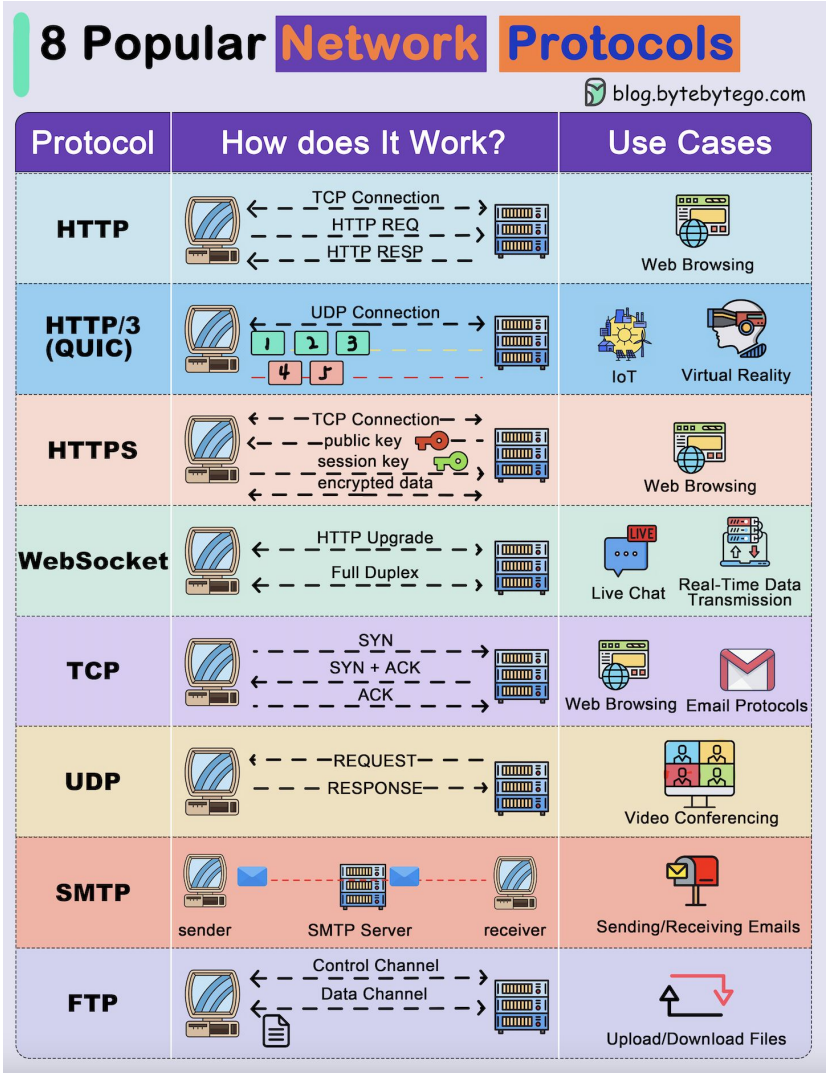
– – –

- Communication over networks must have a certain "structure" so everyone can understand.
- Humans do this all the time - waving, handshakes, clapping. Standard operation procedure that structures how we share info.
- Different "structures" (protocols) are used for different types of communication.

Network Protocols

HTTP is an example of one of the protocols. It is the protocol of the web. The primary protocol you use to access URLs in your web browser.

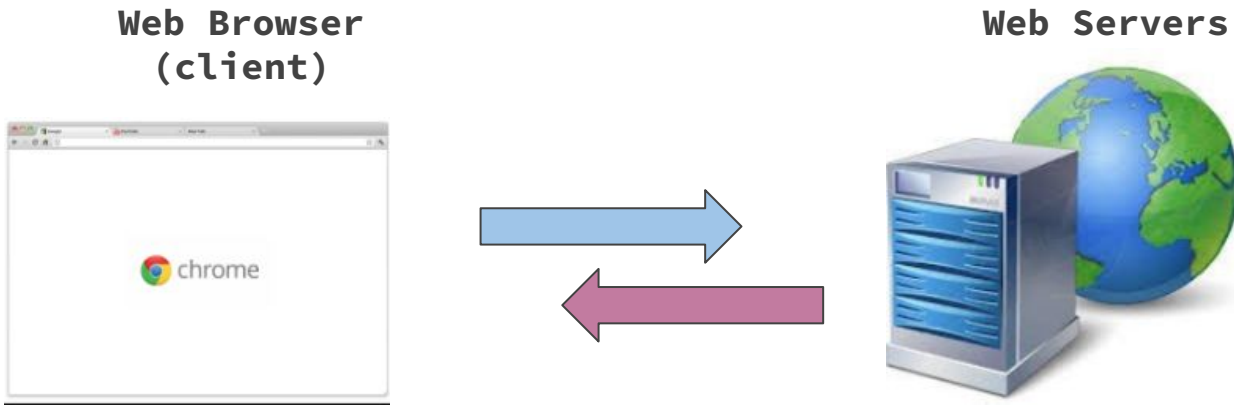
source



HTTP

HTTP: Hypertext Transfer Protocol

i.e. Protocol for sending and receiving HTML documents (nowadays much more)



Web browsers are applications to request and receive HTTP.

NodeJS Express Server

A very popular npm library exists to allow you to run your own HTTP server with NodeJS. It's called Express Server.

Let’s install it (of course)

- **`npm install express cors morgan`**
  - cors to allow access from other domains (needed for frontend to connect)
  - morgan (OPTIONAL) to log (print to terminal) incoming HTTP requests.

`Install the type definitions for the dependencies`

- **`npm install --save-dev @types/express@4.17.21 @types/cors @types/morgan`**

Now we can run our own HTTP server!

NodeJS Express Server

express_basic.ts

```
// Import the Express framework from the 'express' package
import express from 'express';

// Create an Express application instance
const app = express();

// Define the port number the server will listen on
const port = 3000;

// Add built-in middleware to parse incoming requests with text payloads
app.use(express.text());

// Define a GET route for the path '/hello'
app.get('/hello', (req, res) => {
  // Send a plain text response back to the client
  res.send('Hello!');
});

// Define a GET route for the path '/whats/up'
app.get('/whats/up', (req, res) => {
  // Send a JSON string as the response
  res.send(JSON.stringify({
    // Property named "value" with the string "not much"
    value: 'not much',
  }));
});

// Start the server and make it listen on the specified port
app.listen(port, () => {
  // Log a message to the console once the server starts successfully
  console.log(`Listening on port ${port}`);
});
```

What Servers Are Used For

In the past, servers were just used to serve files back to client's browsers.

Nowadays, they're used to quietly exchange large amounts of information back and forth between client and server behind the scenes. Often this happens whilst the server is acting as an API.

This is probably done by sending JSON instead of text/HTML.

Servers Often Respond With JSON

```ts
// Import the Express framework
import express from 'express';

// Create server.
const app = express();

// Define the port number the server will listen on
const port = 3001;

// Add middleware to automatically parse incoming JSON in request bodies
app.use(express.json());

// When someone visits http://localhost:3000/whats/up, this sends a JSON response
app.get('/whats/up', (req, res) => {
  res.json({
    value: 'not much',
  });
});

// Start the server and listen on the specified port
app.listen(port, () => {
  console.log(`Listening on port ${port}`);
});
```

express_json.ts

We can send javascript objects as JSON

How Servers Receive Information

```ts
import express from 'express';

const app = express();
const port = 3000;

app.use(express.json());

app.get('/my/url', (req, res) => {
  const name = req.query.name;
  const age = req.query.age;
  res.json({
    name: `Name is ${name}`,
    age: `Age is ${age}`,
  });
});

app.listen(port, () => {
  console.log(`Listening on port ${port}`);
});
```

input_query.ts

Servers can receive information through the URL. This is called the **query** data.
Eg. Add `?name=Shaveen&age=22` to the end of our url.

How Servers Receive Information

```ts
import express from 'express';

const app = express();
const port = 3001;

app.use(express.json());

app.get('/my/url/:name', (req, res) => {
  const name = req.params.name;
  res.json({
    name: `Name is ${name}`,
  });
});
// app.get(...) tells your server:
// 👉 "When someone makes a GET request to a
// certain path (URL), here's how to respond."


app.listen(port, () => {
  console.log(`Listening on port ${port}`);
});
```

input_params.ts

Servers can receive information through the URL. This is called the **params** data.
Eg. Try `localhost:3000/my/url/Shaveen`

Feedback

Or go to the form here